Table of Contents

1. Identity Lifecycle
2. Credential Governance
3. Incident Response Playbook

1. Identity Lifecycle

Every identity in scope ” human or non-human ” follows the same four stages. An identity with no owner, no review date, or no decommission path is treated as a finding, not an exception.

1.1 Provisioning

Identity type	Who approves	How it's provisioned	Default privilege
Human user	Self-service sign-up	Auth0 Universal Login, TOTP MFA enrolled at first login	`read:ai-data` on own records only
Human admin	IAM Lead	Manual grant in Auth0, MFA mandatory	`write:admin` , logged and reviewed monthly
AI agent (orchestrator)	AI Platform Lead	Registered as an Auth0 application; `agent_id` claim issued via Action	Never exceeds the delegating user's scope
OAuth M2M client	IAM Lead + DevSecOps	Auth0 Machine-to-Machine application, scoped to one API	Least privilege for its one job
LLM API key	AI Platform Lead	Issued by the LLM provider, stored in a secrets vault, never in code or prompts	Scoped to the one model/endpoint needed

Identity type	Who approves	How it's provisioned	Default privilege
RAG service identity	Knowledge Base Owner	Service account, read-only by default	Read on approved indices; write only for ingestion jobs
MCP server identity	Integrations Lead	Per-tool credential, registered in the tool allowlist	One tool per identity where practical

Timeline: provisioning request to active identity, 2 business days for human accounts, 5 business days for a new non-human identity (requires a security review).

1.2 Review

- **Human accounts:** access reviewed every 90 days by the identity's manager; admin-scoped accounts reviewed every 30 days by the IAM Lead.
- **Non-human identities:** reviewed every 90 days by the owning team lead (see table above), confirming the identity is still in use and still holds only the scopes it needs. Unused non-human identities are flagged for decommission after 30 days of no activity.
- Every review is logged with the date, reviewer, and outcome (keep / reduce scope / decommission).

1.3 Rotation

- Human passwords: no fixed rotation (per current NIST guidance, forced periodic password rotation is discouraged); instead, MFA is mandatory and rotation is triggered by a confirmed compromise.
- Non-human credentials: see Section 2, Credential Governance ” this is where the 90-day rotation requirement lives.

1.4 Decommissioning

Trigger	Action	Timeline
Employee offboarding	Auth0 user disabled, sessions revoked, admin grants removed	Same business day

Trigger	Action	Timeline
Non-human identity retired (service decommissioned)	Credentials revoked, Auth0 application deleted, secrets removed from vault	Within 2 business days
Compromised identity (any type)	Immediate suspension, see Incident Response Playbook	Immediate (minutes)
30 days of inactivity (non-human)	Flagged, owner notified, decommissioned if unconfirmed after 7 more days	37 days from last activity

2. Credential Governance

2.1 API key and credential rotation schedule

Credential type	Maximum lifetime before rotation
LLM API key	90 days (hard maximum)
M2M client secret	90 days
RAG service account credential	90 days
User access token	300 seconds (per Project 4 hardening)
M2M access token	60 seconds
Refresh token	Rotating, reuse detection on

No credential in this platform is permitted to exceed a 90-day lifetime. This is a hard maximum, not a target; expired-but-not-yet-rotated credentials are treated as a policy violation, not an acceptable delay.

2.2 Prohibition on long-lived credentials for agents

AI agents, M2M clients, and service identities must never hold a static, non-expiring credential. Every non-human identity authenticates with a short-lived, auto-rotating token obtained through Auth0's client-credentials flow. A long-lived credential discovered in code, configuration, or logs is treated as an active incident (see Section 3, Scenario 1) and triggers immediate revocation, not a scheduled fix.

2.3 Secrets management standards

- Secrets (client secrets, API keys, signing keys) live only in a dedicated secrets vault or environment variables injected at runtime " never in source code, commit history, prompts, or logs.
- Every repository in scope runs automated secret scanning on every commit and pull request; a detected secret blocks the merge until it is revoked and removed.
- Application logs and the `auth.py` audit log are built to never record a raw token or credential value â€" only its identity claims (`sub` , `agent_id` , `jti` , scope), per Project 2's design.
- The LLM's system prompt and any text it retrieves must never contain a live, usable credential. Where a credential-shaped value must be referenced (for testing, as in this capstone's lab), it is clearly marked fake and cannot be used against a real system.

3. Incident Response Playbook

Each scenario below follows the same five-stage structure: **Detection signals** (what tells us this is happening), **Containment** (stop it from getting worse), **Evidence preservation** (what to keep, and how, for the post-incident review), **Notification** (who gets told, and when), **Recovery** (how service returns to normal, safely).

Scenario 1: Leaked LLM API Key

Detection signals

- Anomaly detector (Project 4) fires a `volume_spike` or unexpected-scope alert tied to the LLM provider's usage dashboard, outside expected patterns.
- LLM provider's own billing/usage alert shows a spend or request-rate spike.
- The key appears in a public location: a secret-scanning alert on a commit, a paste site, or a guardrail redaction log showing the key was present in a model response (Project 4's output guardrail should have caught and redacted it before it left the system " a leak means that control was bypassed or missing).

Containment

1. Revoke the leaked key immediately at the LLM provider's dashboard. This is the single highest-priority action; everything else follows it.
2. Issue a new key through the same provisioning process as Section 1.1, store it in the secrets vault, and redeploy.

3. Disable the affected service or route (fail closed, not open) if the key cannot be rotated within minutes.

Evidence preservation

- Export the LLM provider's usage logs for the suspected compromise window before they age out of the provider's retention period.
- Preserve the audit log (`auth.py` / `detect_anomalies.py` output) covering the incident window; copy it out of the live system so it can't be altered.
- Record exactly when and how the key was discovered (which detection signal fired first), since this feeds the root-cause analysis.

Notification

- AI Platform Lead and Security Operations Lead: immediately.
- IAM Lead: within 1 hour.
- Affected customers, legal/compliance: only if the investigation confirms customer data was actually exposed, per SecureNova's standard data-incident process and applicable breach-notification law; this determination is made by Security Operations, not assumed by default.

Recovery

1. Confirm the new key works and the old one returns `401` / `invalid_api_key` on any further use.
2. Review how the key leaked (code, logs, prompt injection) and close that specific path â€” this is where the Project 4 output guardrail's effectiveness is reassessed if that was the leak vector.
3. Post-incident review within 5 business days; update this policy if a gap is found.

Scenario 2: Compromised Agent Identity

Detection signals

- `scope_change` anomaly alert: the same agent identity suddenly presents a different, broader scope than its normal pattern.
- Agent B (or any downstream agent) receives and acts on a message whose sender cannot be cryptographically verified (Project 4's Ed25519 signing should reject this before it reaches here — detection here means that control was skipped, disabled, or the message slipped through before the fix was deployed).
- A privileged tool (`issue_refund` , `export_customer_data`) is invoked outside normal business hours or at a rate inconsistent with real usage.

Containment

1. Suspend the affected agent's Auth0 application (revoke its client credentials immediately ” this invalidates its ability to obtain new tokens).
2. Revoke any outstanding tokens issued to that identity where the provider supports it; short token lifetimes (Project 4 hardening) limit the exposure window regardless.
3. If the compromise is inter-agent (Agent B accepted a spoofed orchestrator message), temporarily require human approval for that agent's privileged tool calls until signed-message verification is confirmed active.

Evidence preservation

- Preserve `logs/interactions.jsonl` and `logs/tool_call_log` entries for the affected agent, covering the full suspected window.
- Preserve the specific message(s) that triggered the privileged action, including the claimed sender and (if present) its signature/verification result.

Notification

- AI Platform Lead and IAM Lead: immediately.
- Affected business stakeholders (e.g. if a real refund or data export occurred): within 4 hours, once scope of impact is understood.

Recovery

1. Re-provision the agent identity following Section 1.1, with a fresh credential.
2. Confirm Ed25519 message signing is enforced for every privileged tool call before returning the agent to normal operation ” this is the control that should prevent recurrence.
3. Review whether the agent's default privilege level was broader than it needed to be; reduce scope where possible (least privilege).

Scenario 3: Successful Prompt Injection Leading to Data Exfiltration

Detection signals

- Output guardrail log shows a redaction event (a credential or JWT-shaped string was caught) ” this is itself a near-miss worth investigating even if nothing actually leaked.

- A customer or internal reviewer reports an unexpected or out-of-character agent response.
- Anomaly detector flags unusual data volume returned from a single `/chat` interaction, or a `TOOL_CALL` invoking `export_customer_data` without a corresponding legitimate business request.

Containment

1. Suspend the `/chat` endpoint or the specific conversation session immediately to stop further exfiltration.
2. Identify and remove the poisoned content from the RAG knowledge base (or the injected input) that caused the behaviour.
3. Rotate any credential that may have been exposed in the exfiltrated output, per Scenario 1, even if the guardrail appears to have redacted it " treat an observed redaction event as a near-miss requiring the same rotation as a confirmed leak, out of caution.

Evidence preservation

- Preserve the full conversation transcript (`logs/interactions.jsonl`): the injected input, the retrieved document(s), and the model's complete response.
- Preserve the specific knowledge-base document or input that carried the injection, unmodified, for forensic review.
- Note which guardrail (input pattern match, output redaction) did or did not fire, and why " this is the input to the post-incident control review.

Notification

- Security Operations Lead and AI Platform Lead: immediately.
- Data Protection / Legal: within 4 hours if any customer or personal data is confirmed in the exfiltrated output, to begin breach-notification assessment.
- Customers: only following Legal's determination, per standard breach-notification obligations and timelines in the applicable jurisdiction.

Recovery

1. Remove or patch the poisoned knowledge-base content; re-review all documents in that source for similar content before reopening the source.
2. Confirm the input guardrail pattern that should have caught this is updated to cover the technique used, and add a regression test for it.

3. Post-incident review within 5 business days, specifically assessing whether the injection technique used reveals a gap the Project 3 red-team exercise should have found — update the red-team test suite accordingly.